

Министерство Образования Республики Беларусь
УО Брестский Государственный Технический Университет
Кафедра интеллектуальных информационных технологий

Лабораторная работа № 7

По дисциплине: «Современные платформы программирования»

За 6 семестр

Выполнил:

Студент 3 курса

Группы ПО-12

Матюх И.М.

Проверил:

Кулик А.Д.

Цель: освоить возможности языка программирования Python в разработке оконных приложений.

Вариант-12

Задание 1

Построение графических примитивов и надписей

Требования к выполнению

- Реализовать соответствующие классы, указанные в задании;
- Организовать ввод параметров для создания объектов (использовать экранные компоненты);
- Осуществить визуализацию графических примитивов

Важное замечание: должна быть предусмотрена возможность приостановки выполнения визуализации, изменения параметров «на лету» и снятия скриншотов с сохранением в текущую активную директорию.

Для всех динамических сцен необходимо задавать параметр скорости!

12) Задать составление строки из символов, появляющихся из разных углов формы и выстраивающихся друг за другом. Процесс должен циклически повторяться.

Задание 2

Реализовать построение заданного типа фрактала по варианту

Везде, где это необходимо, предусмотреть ввод параметров, влияющих на внешний вид фрактала.

12) Кривая Гильберта

Код программы

Task1.py

```
import tkinter as tk
from tkinter import ttk, filedialog, messagebox
from dataclasses import dataclass
import math
from datetime import datetime
import os

try:
    from PIL import Image, ImageGrab
    PIL_AVAILABLE = True
except ImportError:
    PIL_AVAILABLE = False
    messagebox.showwarning("Предупреждение",
```

"Для сохранения в PNG установите Pillow: pip install Pillow\n"
"Скриншоты будут сохраняться в EPS формате")

```
@dataclass
```

```
class TextAnimationConfig:
```

```
    text: str
```

```
    speed: int
```

```
    form_rect: tuple
```

```
@dataclass
```

```
class HilbertConfig:
```

```
    order: int
```

```
    size: int
```

```
    start_x: int
```

```
    start_y: int
```

```
class AnimatedTextString:
```

```
    def __init__(self, canvas, config):
```

```
        self.canvas = canvas
```

```
        self.config = config
```

```
        self.current_display_text = ""
```

```
        self.current_index = 0
```

```
        self.target_positions = []
```

```
        self.animation_running = False
```

```
        self.after_id = None
```

```
        self.paused = False
```

```
        self.auto_restart = True
```

```
        self.corners = [
```

```
            (config.form_rect[0], config.form_rect[1]),
```

```
            (config.form_rect[2], config.form_rect[1]),
```

```
            (config.form_rect[0], config.form_rect[3]),
```

```
            (config.form_rect[2], config.form_rect[3])
```

```
        ]
```

```
        self.calculate_target_positions()
```

```
        self.draw_form()
```

```
    def calculate_target_positions(self):
```

```
        char_width = 35
```

```
        text = self.config.text
```

```

rect = self.config.form_rect

center_x = (rect[0] + rect[2]) // 2
center_y = (rect[1] + rect[3]) // 2
start_x = center_x - (len(text) * char_width) // 2
y = center_y

for i, char in enumerate(text):
    target_x = start_x + i * char_width
    target_y = y
    corner = self.corners[i % 4]
    self.target_positions.append({
        'char': char,
        'target': (target_x, target_y),
        'start': corner,
        'current_pos': list(corner),
        'progress': 0
    })

def draw_form(self):
    self.canvas.delete("form")
    rect = self.config.form_rect
    self.canvas.create_rectangle(
        rect[0], rect[1], rect[2], rect[3],
        outline="#6666cc", width=3, fill="", tags="form"
    )

    colors = ["#ff6666", "#66ff66", "#6666ff", "#ffff66"]
    for i, corner in enumerate(self.corners):
        self.canvas.create_oval(
            corner[0]-5, corner[1]-5,
            corner[0]+5, corner[1]+5,
            fill=colors[i], outline="black", width=2, tags="form"
        )

def draw_static_text(self):
    self.canvas.delete("static_text")
    char_width = 35
    rect = self.config.form_rect
    center_x = (rect[0] + rect[2]) // 2
    center_y = (rect[1] + rect[3]) // 2
    start_x = center_x - (len(self.config.text) * char_width) // 2
    y = center_y

    for i, char in enumerate(self.current_display_text):
        x = start_x + i * char_width

```

```

        self.canvas.create_text(
            x, y, text=char,
            font=("Arial", 32, "bold"),
            fill="#2ecc71", tags="static_text"
        )

def start_animation(self):
    if not self.animation_running:
        self.animation_running = True
        self.current_index = 0
        self.current_display_text = ""
        self.draw_form()
        self.draw_static_text()
        self.animate_next_char()

def stop_animation(self):
    if self.after_id:
        self.canvas.after_cancel(self.after_id)
    self.animation_running = False

def animate_next_char(self):
    if self.paused:
        self.after_id = self.canvas.after(100, self.animate_next_char)
        return

    if self.current_index >= len(self.target_positions):
        self.animation_running = False
        if self.auto_restart:
            self.after_id = self.canvas.after(2000, self.start_animation)
        if hasattr(self, 'on_complete'):
            self.on_complete()
        return

    pos_data = self.target_positions[self.current_index]
    start_x, start_y = pos_data['start']
    target_x, target_y = pos_data['target']

    char_id = self.canvas.create_text(
        start_x, start_y,
        text=pos_data['char'],
        font=("Arial", 36, "bold"),
        fill="#ff6600",
        tags="animated_char"
    )

    steps = 40

```

```

step = 0

def move_step():
    nonlocal step
    if self.paused:
        self.after_id = self.canvas.after(50, move_step)
        return

    if step <= steps:
        progress = step / steps
        x = start_x + (target_x - start_x) * progress
        y = start_y + (target_y - start_y) * progress
        self.canvas.coords(char_id, x, y)

        r = int(255 * (1 - progress))
        g = int(255 * progress)
        b = int(100)
        color = f"#{r:02x}{g:02x}{b:02x}"
        self.canvas.itemconfig(char_id, fill=color)

        step += 1
        self.after_id = self.canvas.after(
            int(self.config.speed / steps), move_step
        )
    else:
        self.current_display_text += pos_data['char']
        self.current_index += 1
        self.draw_static_text()
        self.canvas.delete(char_id)
        self.animate_next_char()

move_step()

def draw_complete(self):
    self.stop_animation()
    self.canvas.delete("animated_char")
    self.current_display_text = self.config.text
    self.current_index = len(self.config.text)
    self.draw_static_text()

def reset(self):
    self.stop_animation()
    self.canvas.delete("animated_char")
    self.canvas.delete("static_text")
    self.current_display_text = ""
    self.current_index = 0

```

```

        self.paused = False
        self.animation_running = False
        self.draw_form()

    def toggle_pause(self):
        self.paused = not self.paused
        return not self.paused

    def set_auto_restart(self, value):
        self.auto_restart = value

class HilbertCurve:

    def __init__(self, canvas, config):
        self.canvas = canvas
        self.config = config
        self.points = []
        self.lines = []
        self.current_point = 0
        self.animation_running = False
        self.after_id = None
        self.paused = False
        self.generate_curve()

    def generate_curve(self):
        self.points = []
        n = 2 ** self.config.order
        step = self.config.size / n

        def hilbert(x, y, xi, xj, yi, yj, n):
            if n <= 1:
                px = self.config.start_x + step * (x + (xi + yi) / 2)
                py = self.config.start_y + step * (y + (xj + yj) / 2)
                self.points.append((px, py))
            else:
                hilbert(x, y, yi//2, yj//2, xi//2, xj//2, n//2)
                hilbert(x + xi//2, y + xj//2, xi//2, xj//2, yi//2, yj//2, n//2)
                hilbert(x + xi//2 + yi//2, y + xj//2 + yj//2, xi//2, xj//2, yi//2, yj//2, n//2)
                hilbert(x + yi//2, y + yj//2, -yi//2, -yj//2, -xi//2, -xj//2, n//2)

        hilbert(0, 0, n, 0, 0, n, n)

    def draw_static(self):
        self.clear()
        for i in range(len(self.points) - 1):

```

```

        progress = i / len(self.points)
        r = int(100 + 155 * progress)
        g = int(50 + 150 * (1 - progress))
        b = int(200 - 100 * progress)
        color = f"#{r:02x}{g:02x}{b:02x}"
        line = self.canvas.create_line(
            self.points[i][0], self.points[i][1],
            self.points[i+1][0], self.points[i+1][1],
            fill=color, width=2, tags="fractal_line"
        )
        self.lines.append(line)

def start_animation(self):
    if not self.animation_running:
        self.current_point = 0
        self.animation_running = True
        self.draw_next_segment()

def draw_next_segment(self):
    if self.paused:
        self.after_id = self.canvas.after(50, self.draw_next_segment)
        return

    if self.current_point >= len(self.points) - 1:
        self.animation_running = False
        if hasattr(self, 'on_complete'):
            self.on_complete()
        return

    i = self.current_point
    progress = i / len(self.points)
    r = int(100 + 155 * progress)
    g = int(50 + 150 * (1 - progress))
    b = int(200 - 100 * progress)
    color = f"#{r:02x}{g:02x}{b:02x}"

    line = self.canvas.create_line(
        self.points[i][0], self.points[i][1],
        self.points[i+1][0], self.points[i+1][1],
        fill=color, width=2, tags="fractal_line"
    )
    self.lines.append(line)
    self.current_point += 1

    delay = int(50 / (self.config.order + 1))
    self.after_id = self.canvas.after(delay, self.draw_next_segment)

```



```

def stop_animation(self):
    if self.after_id:
        self.canvas.after_cancel(self.after_id)
    self.animation_running = False

def clear(self):
    self.stop_animation()
    for line in self.lines:
        self.canvas.delete(line)
    self.lines.clear()
    self.current_point = 0

def reset(self):
    self.clear()
    self.paused = False

def toggle_pause(self):
    self.paused = not self.paused
    return not self.paused

class GraphicsApp:

    def __init__(self, root):
        self.root = root
        self.root.title("Графические примитивы и фракталы")
        self.root.geometry("1000x750")

        self.text_animation = None
        self.hilbert_curve = None
        self.ui_elements = {}

        self.setup_ui()

    def setup_ui(self):
        control_frame = ttk.Frame(self.root)
        control_frame.pack(side=tk.TOP, fill=tk.X, padx=5, pady=5)

        notebook = ttk.Notebook(self.root)
        notebook.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

        self.ui_elements["tabs"] = {}
        self.ui_elements["tabs"]["text"] = ttk.Frame(notebook)
        self.ui_elements["tabs"]["hilbert"] = ttk.Frame(notebook)
        notebook.add(self.ui_elements["tabs"]["text"], text="Анимированная строка")

```

```

notebook.add(self.ui_elements["tabs"]["hilbert"], text="Кривая Гильберта")

self.setup_text_tab()
self.setup_hilbert_tab()
self.setup_control_buttons(control_frame)

def setup_text_tab(self):
    parent = self.ui_elements["tabs"]["text"]

    input_frame = ttk.LabelFrame(parent, text="Параметры анимации строки")
    input_frame.pack(side=tk.TOP, fill=tk.X, padx=5, pady=5)

    ttk.Label(input_frame, text="Текст:").grid(row=0, column=0, padx=5, pady=5, sticky=tk.W)
    self.ui_elements["controls"] = {}
    self.ui_elements["controls"]["text_string"] = ttk.Entry(input_frame, width=20,
font=("Arial", 12))
    self.ui_elements["controls"]["text_string"].insert(0, "PYTHON")
    self.ui_elements["controls"]["text_string"].grid(row=0, column=1, padx=5, pady=5)

    ttk.Label(input_frame, text="Скорость (мс/символ):").grid(row=0, column=2, padx=5,
pady=5, sticky=tk.W)
    self.ui_elements["controls"]["text_speed"] = ttk.Spinbox(input_frame, from_=20, to=300,
width=10)
    self.ui_elements["controls"]["text_speed"].set(80)
    self.ui_elements["controls"]["text_speed"].grid(row=0, column=3, padx=5, pady=5)

    ttk.Button(input_frame, text="Создать", command=self.create_text_animation).grid(row=0,
column=4, padx=5, pady=5)

    info_frame = ttk.LabelFrame(parent, text="Информация")
    info_frame.pack(side=tk.TOP, fill=tk.X, padx=5, pady=5)

    self.ui_elements["text_progress"] = ttk.Label(info_frame, text="Прогресс: 0/0")
    self.ui_elements["text_progress"].pack(side=tk.LEFT, padx=10)
    self.ui_elements["text_status"] = ttk.Label(info_frame, text="Статус: Готов")
    self.ui_elements["text_status"].pack(side=tk.LEFT, padx=10)

    self.ui_elements["auto_restart"] = tk.BooleanVar(value=True)
    ttk.Checkbutton(
        info_frame, text="Автоповтор",
        variable=self.ui_elements["auto_restart"],
        command=self.toggle_auto_restart
    ).pack(side=tk.LEFT, padx=10)

    canvas_frame = ttk.Frame(parent)
    canvas_frame.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

```

```

self.ui_elements["canvases"] = {}
self.ui_elements["canvases"]["text"] = tk.Canvas(canvas_frame, bg="white", width=800,
height=500)
self.ui_elements["canvases"]["text"].pack(fill=tk.BOTH, expand=True)

text_anim_frame = ttk.Frame(parent)
text_anim_frame.pack(side=tk.BOTTOM, fill=tk.X, padx=5, pady=5)

ttk.Button(text_anim_frame, text="Анимация",
command=self.start_text_animation).pack(side=tk.LEFT, padx=5)
ttk.Button(text_anim_frame, text="Статично",
command=self.draw_text_static).pack(side=tk.LEFT, padx=5)
ttk.Button(text_anim_frame, text="Сброс",
command=self.reset_text_animation).pack(side=tk.LEFT, padx=5)

def setup_hilbert_tab(self):
    parent = self.ui_elements["tabs"]["hilbert"]

    input_frame = ttk.LabelFrame(parent, text="Параметры кривой Гильберта")
    input_frame.pack(side=tk.TOP, fill=tk.X, padx=5, pady=5)

    ttk.Label(input_frame, text="Порядок (1-6):").grid(row=0, column=0, padx=5, pady=5,
sticky=tk.W)
    self.ui_elements["controls"]["hilbert_order"] = ttk.Spinbox(input_frame, from_=1, to=6,
width=10)
    self.ui_elements["controls"]["hilbert_order"].set(3)
    self.ui_elements["controls"]["hilbert_order"].grid(row=0, column=1, padx=5, pady=5)

    ttk.Label(input_frame, text="Размер:").grid(row=0, column=2, padx=5, pady=5,
sticky=tk.W)
    self.ui_elements["controls"]["hilbert_size"] = ttk.Spinbox(input_frame, from_=200,
to=600, width=10)
    self.ui_elements["controls"]["hilbert_size"].set(450)
    self.ui_elements["controls"]["hilbert_size"].grid(row=0, column=3, padx=5, pady=5)

    ttk.Button(input_frame, text="Построить", command=self.create_hilbert_curve).grid(row=0,
column=4, padx=5, pady=5)

    info_frame = ttk.LabelFrame(parent, text="Информация")
    info_frame.pack(side=tk.TOP, fill=tk.X, padx=5, pady=5)

    self.ui_elements["hilbert_points"] = ttk.Label(info_frame, text="Точек: 0")
    self.ui_elements["hilbert_points"].pack(side=tk.LEFT, padx=10)
    self.ui_elements["hilbert_status"] = ttk.Label(info_frame, text="Статус: Готов")
    self.ui_elements["hilbert_status"].pack(side=tk.LEFT, padx=10)

```

```

canvas_frame = ttk.Frame(parent)
canvas_frame.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

self.ui_elements["canvases"]["hilbert"] = tk.Canvas(canvas_frame, bg="white", width=800,
height=500)
self.ui_elements["canvases"]["hilbert"].pack(fill=tk.BOTH, expand=True)

hilbert_anim_frame = ttk.Frame(parent)
hilbert_anim_frame.pack(side=tk.BOTTOM, fill=tk.X, padx=5, pady=5)

    ttk.Button(hilbert_anim_frame, text="Анимация",
command=self.start_hilbert_animation).pack(side=tk.LEFT, padx=5)
    ttk.Button(hilbert_anim_frame, text="Статично",
command=self.draw_hilbert_static).pack(side=tk.LEFT, padx=5)
    ttk.Button(hilbert_anim_frame, text="Сброс",
command=self.reset_hilbert).pack(side=tk.LEFT, padx=5)

def setup_control_buttons(self, frame):
    ttk.Button(frame, text="Пауза/Продолжить", command=self.toggle_pause).pack(side=tk.LEFT,
padx=5)
    ttk.Button(frame, text="Скриншот", command=self.take_screenshot).pack(side=tk.LEFT,
padx=5)
    ttk.Button(frame, text="Выход", command=self.root.quit).pack(side=tk.RIGHT, padx=5)

def toggle_auto_restart(self):
    if self.text_animation:
        self.text_animation.set_auto_restart(self.ui_elements["auto_restart"].get())

def create_text_animation(self):
    text = self.ui_elements["controls"]["text_string"].get()
    if not text:
        text = "PYTHON"
    speed = int(self.ui_elements["controls"]["text_speed"].get())
    rect = (100, 80, 700, 380)

    config = TextAnimationConfig(text, speed, rect)
    canvas = self.ui_elements["canvases"]["text"]
    canvas.delete("all")

    self.text_animation = AnimatedTextString(canvas, config)
    self.text_animation.set_auto_restart(self.ui_elements["auto_restart"].get())
    self.text_animation.on_complete = self.on_text_complete

    self.ui_elements["text_status"].config(text="Статус: Создано")
    self.ui_elements["text_progress"].config(text=f"Прогресс: 0/{len(text)}")

```

```

def start_text_animation(self):
    if self.text_animation:
        self.text_animation.start_animation()
        self.ui_elements["text_status"].config(text="Статус: Анимация")
        self.update_text_info()

def draw_text_static(self):
    if self.text_animation:
        self.text_animation.stop_animation()
        self.text_animation.draw_complete()
        self.ui_elements["text_status"].config(text="Статус: Статично")
        self.ui_elements["text_progress"].config(
            text=f"Порпец:
{len(self.text_animation.config.text)}/{len(self.text_animation.config.text)}"
        )

def reset_text_animation(self):
    if self.text_animation:
        self.text_animation.reset()
        self.ui_elements["text_status"].config(text="Статус: Сброшено")
        self.ui_elements["text_progress"].config(text=f"Порпец:
0/{len(self.text_animation.config.text)}")

def on_text_complete(self):
    self.ui_elements["text_status"].config(text="Статус: Завершено")
    self.ui_elements["text_progress"].config(
        text=f"Порпец:
{len(self.text_animation.config.text)}/{len(self.text_animation.config.text)}"
    )

def update_text_info(self):
    if self.text_animation and self.text_animation.animation_running:
        current = self.text_animation.current_index
        total = len(self.text_animation.config.text)
        self.ui_elements["text_progress"].config(text=f"Порпец: {current}/{total}")
        self.root.after(100, self.update_text_info)

def create_hilbert_curve(self):
    order = int(self.ui_elements["controls"]["hilbert_order"].get())
    size = int(self.ui_elements["controls"]["hilbert_size"].get())

    canvas = self.ui_elements["canvases"]["hilbert"]
    canvas_width = canvas.winfo_width() if canvas.winfo_width() > 100 else 800
    canvas_height = canvas.winfo_height() if canvas.winfo_height() > 100 else 500

```

```

start_x = (canvas_width - size) // 2
start_y = (canvas_height - size) // 2 + 80

config = HilbertConfig(order, size, start_x, start_y)
canvas.delete("all")

self.hilbert_curve = HilbertCurve(canvas, config)
self.hilbert_curve.on_complete = self.on_hilbert_complete

points = len(self.hilbert_curve.points)
self.ui_elements["hilbert_points"].config(text=f"Точек: {points}")
self.ui_elements["hilbert_status"].config(text="Статус: Создано")

def start_hilbert_animation(self):
    if self.hilbert_curve:
        self.hilbert_curve.clear()
        self.hilbert_curve.start_animation()
        self.ui_elements["hilbert_status"].config(text="Статус: Анимация")

def draw_hilbert_static(self):
    if self.hilbert_curve:
        self.hilbert_curve.clear()
        self.hilbert_curve.draw_static()
        self.ui_elements["hilbert_status"].config(text="Статус: Статично")

def reset_hilbert(self):
    if self.hilbert_curve:
        self.hilbert_curve.reset()
        self.ui_elements["hilbert_status"].config(text="Статус: Сброшено")

def on_hilbert_complete(self):
    self.ui_elements["hilbert_status"].config(text="Статус: Завершено")

def toggle_pause(self):
    notebook = self.root.nametowidget(self.root.winfo_children()[1])
    current_tab = notebook.index(notebook.select())

    if current_tab == 0:
        if self.text_animation and self.text_animation.animation_running:
            self.text_animation.toggle_pause()
            if self.text_animation.paused:
                self.ui_elements["text_status"].config(text="Статус: Пауза")
            else:
                self.ui_elements["text_status"].config(text="Статус: Анимация")
    else:
        if self.hilbert_curve and self.hilbert_curve.animation_running:

```

```

        self.hilbert_curve.toggle_pause()
    if self.hilbert_curve.paused:
        self.ui_elements["hilbert_status"].config(text="Статус: Пауза")
    else:
        self.ui_elements["hilbert_status"].config(text="Статус: Анимация")

def take_screenshot(self):
    try:
        notebook = self.root.nametowidget(self.root.winfo_children()[1])
        current_tab = notebook.index(notebook.select())

        canvas = self.ui_elements["canvases"]["text"] if current_tab == 0 else \
            self.ui_elements["canvases"]["hilbert"]

        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        default_name = f"screenshot_{timestamp}.png"

        file_path = filedialog.asksaveasfilename(
            defaultextension=".png",
            initialfile=default_name,
            filetypes=[
                ("PNG files", "*.png"),
                ("All files", "*.*")
            ],
            title="Сохранить скриншот"
        )

        if not file_path:
            return

        if PIL_AVAILABLE:
            x = canvas.winfo_rootx()
            y = canvas.winfo_rooty()
            width = canvas.winfo_width()
            height = canvas.winfo_height()

            screenshot = ImageGrab.grab(bbox=(x, y, x + width, y + height))
            screenshot.save(file_path, "PNG")
            messagebox.showinfo("Скриншот", f"Сохранено в PNG:\n{file_path}")
        else:
            eps_file = file_path.replace('.png', '.eps')
            canvas.postscript(
                file=eps_file,
                colormode="color",
                width=canvas.winfo_width(),
                height=canvas.winfo_height()
            )
    
```

```

    )

    try:
        img = Image.open(eps_file)
        img.save(file_path, "PNG")
        os.remove(eps_file)
        messagebox.showinfo("Скриншот", f"Сохранено в PNG:\n{file_path}")
    except:
        messagebox.showwarning("Скриншот",
                                f"Сохранено в EPS (установите Pillow для PNG):\n{eps_file}")

except Exception as err:
    messagebox.showerror("Ошибка", f"Не удалось сохранить скриншот:\n{str(err)}")

def main():
    root = tk.Tk()

    if not PIL_AVAILABLE:
        warning = tk.Toplevel(root)
        warning.title("Предупреждение")
        warning.geometry("400x150")
        warning.transient(root)
        warning.grab_set()

        label = tk.Label(warning,
                          text="Для сохранения скриншотов в PNG необходимо установить Pillow:\n\n"
                                "pip install Pillow\n\n"
                                "Скриншоты будут сохраняться в EPS формате.",
                          justify=tk.LEFT, padx=20, pady=20)
        label.pack()

        tk.Button(warning, text="OK", command=warning.destroy).pack(pady=10)

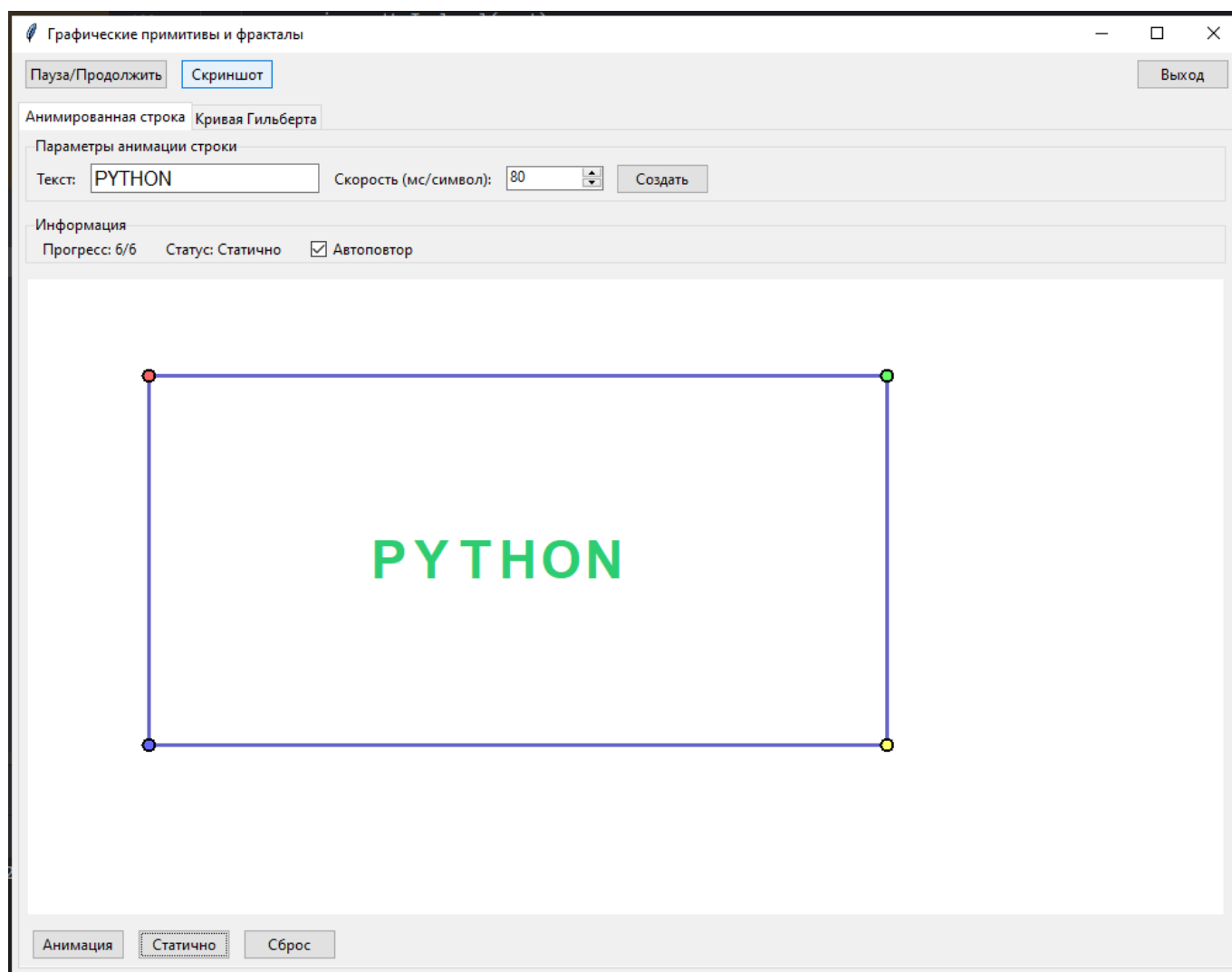
        root.wait_window(warning)

    app = GraphicsApp(root)
    root.mainloop()

if __name__ == "__main__":
    main()

```


Результат выполнения программы:



Пауза/Продолжить

Скриншот

Анимированная строка

Кривая Гильберта

Параметры кривой Гильберта

Порядок (1-6):

3

Размер:

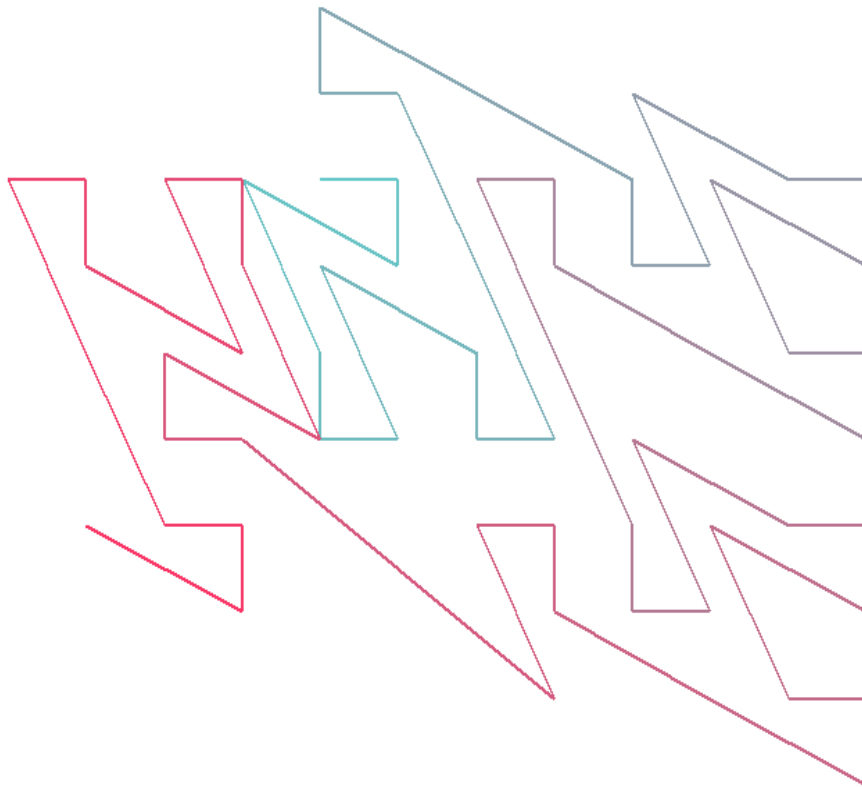
450

Построить

Информация

Точек: 64

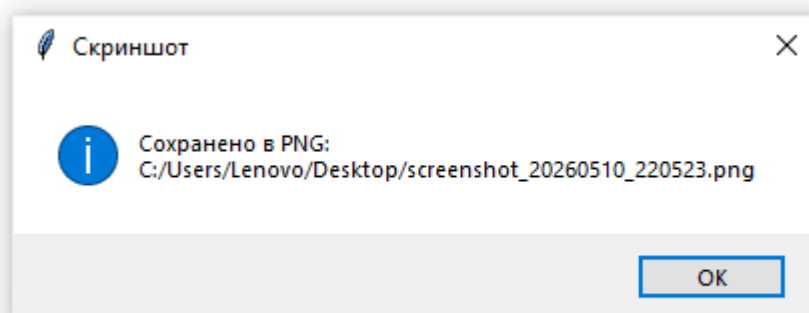
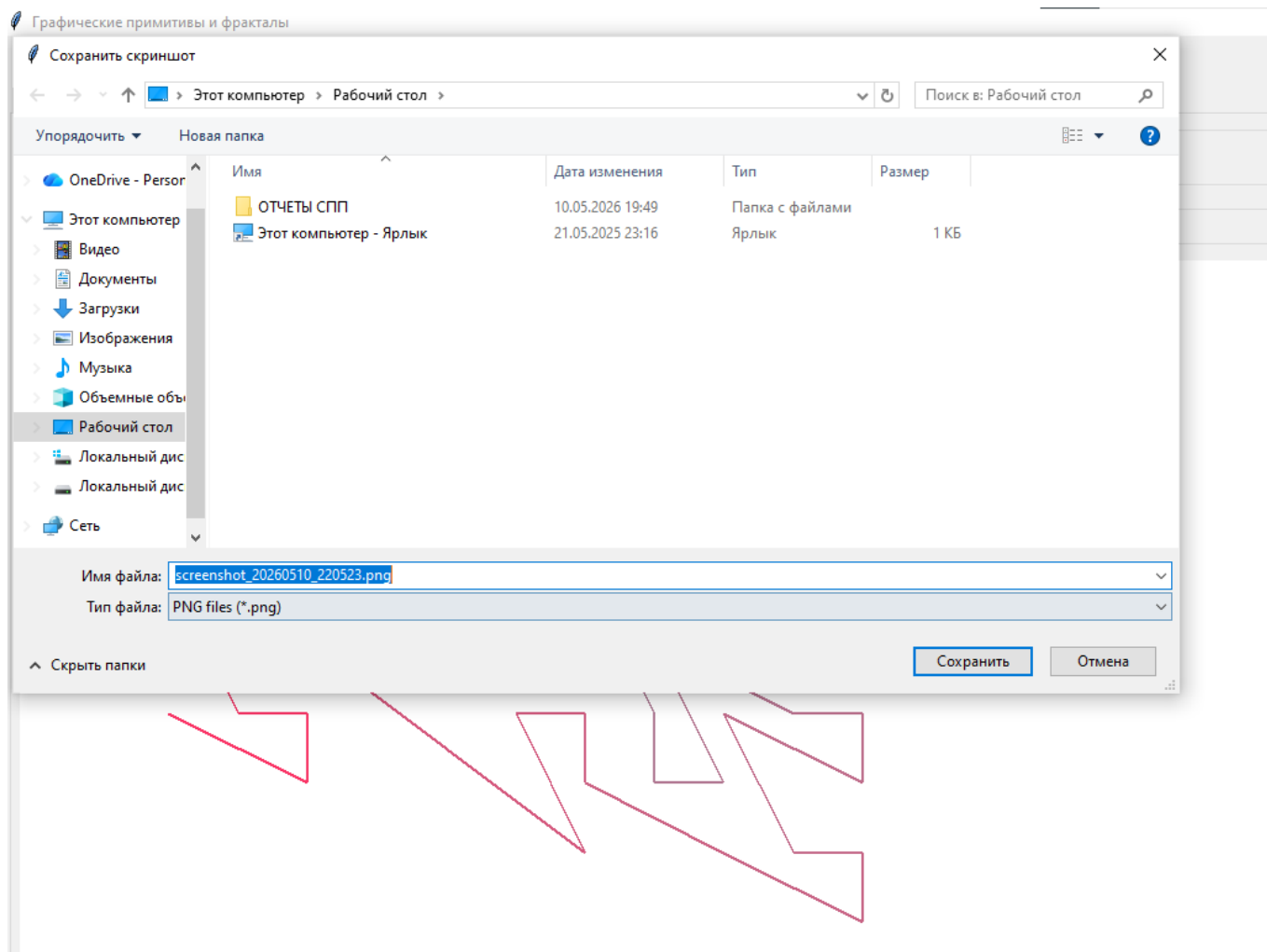
Статус: Статично



Анимация

Статично

Сброс



Вывод: освоил возможности языка программирования Python в разработке оконных приложений.